

TRANSITION REPORT: AWS COGNITO TO DATABASE JWT

Project: BSSC Backend Portal (Cloned from JSSC Production)
Date: June 30, 2026

1. Overview of Transition

The BSSC Backend Project has been decoupled from AWS Cognito. All identity lookup, validation, password hashing, and session management are now handled locally within the PostgreSQL database and locally-signed JSON Web Tokens (JWT). This eliminates dependencies on AWS Cognito SDKs, JWKS URL verification, and external trigger setups.

2. Summary of Changes

- `src/utils/jwt.ts` (Added): New local JWT utility file. Uses `bcryptjs` for password hashing and `jsonwebtoken` for token operations.
- `src/middleware/auth.ts` (Modified): Refactored auth middleware to decode local JWT and query users by ID from local PostgreSQL.
- `src/services/auth.service.ts` (Modified): Refactored login, register, candidateRegister, token refresh, forgot/reset password, and logout methods.
- `src/database/schema.ts` (Modified): Simplified users table schema by removing Cognito Sub ID columns and unique indexes.
- `src/utils/cognito.ts` (Removed): Deleted legacy AWS Cognito provider integration helper.
- Cognito Trigger functions (Removed): Removed `customMessage` and `customSmsSender` trigger handlers and serverless trigger mappings.

A. Local JWT Utility (src/utls/jwt.ts)

```
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';
import config from '../config';

export interface JwtPayload {
  userId: string;
  email: string;
  roles: string[];
}

export const hashPassword = async (password: string): Promise<string> => {
  return bcrypt.hash(password, 10);
};

export const comparePassword = async (password: string, hash: string): Promise<boolean> => {
  return bcrypt.compare(password, hash);
};

export const generateTokens = (userId: string, email: string, roles: string[]) => {
  const accessToken = jwt.sign(
    { userId, email, roles },
    config.JWT_SECRET,
    { expiresIn: '1h' }
  );

  const refreshToken = jwt.sign(
    { userId },
    config.JWT_SECRET,
    { expiresIn: '30d' }
  );

  return {
    accessToken,
    refreshToken,
    expiresIn: 3600,
  };
};

export const verifyJwt = (token: string): JwtPayload => {
  try {
    return jwt.verify(token, config.JWT_SECRET) as JwtPayload;
  } catch (err) {
    throw new Error('Your session has expired. Please log in again to continue.');
```

B. Auth Middleware (src/middleware/auth.ts)

```
import type { APIGatewayProxyEventV2 } from 'aws-lambda';
import { UnauthorizedError, ForbiddenError } from '../errors/AppError';
import { verifyJwt, type JwtPayload } from '../utils/jwt';
import { getAuthorizationToken } from '../helpers/request';
import { userRepository } from '../repositories/user.repository';

export interface AuthenticatedUser {
  userId: string;
  email: string;
  roles: string[];
}

export const authenticate = async (event: APIGatewayProxyEventV2): Promise<AuthenticatedUser> => {
  const token = getAuthorizationToken(event);
  if (!token)
    throw new UnauthorizedError(
      'The username or password you entered is incorrect. Please check your details and try again.'
    );

  let decoded: JwtPayload;
  try {
    decoded = verifyJwt(token);
  } catch (err: unknown) {
    throw new UnauthorizedError((err as Error).message);
  }
  console.log('Decoded JWT payload:', decoded);

  /* COGNITO FLOW (REMOVED):
  let userId = decoded.sub;
  let email = decoded.email;
  if (userRepository && typeof userRepository.findByCognitoSubId === 'function') {
    const dbUser = await userRepository.findByCognitoSubId(decoded.sub);
    if (!dbUser) throw new UnauthorizedError('User not found');
    if (!dbUser.isActive) throw new UnauthorizedError('User deactivated');
    userId = dbUser.id;
    email = dbUser.email;
  }
  return {
    userId,
    email,
    roles: decoded['cognito:groups'] || [],
  };
  */

  const dbUser = await userRepository.findById(decoded.userId);
  if (!dbUser) {
    throw new UnauthorizedError(
      'We could not find an account with these details. Please register a new account or check your information.'
    );
  }
  if (!dbUser.isActive) {
    throw new UnauthorizedError(
      'Your account is currently deactivated. Please contact our support team for assistance.'
    );
  }

  return {
    userId: dbUser.id,
    email: dbUser.email,
    roles: decoded.roles || [],
  };
};

export const requireRole = (user: AuthenticatedUser, allowedRoles: string[]): void => {
```

```
console.log(user.roles, 'awanish');
const hasRole = user.roles.some((role) => allowedRoles.includes(role));
if (!hasRole) {
  throw new ForbiddenError(
    `Access requires one of the following roles: ${allowedRoles.join(', ')}`
  );
}
};

export const authenticateUser = authenticate;
export default authenticate;
```

C. Simplified User DB Schema (src/database/schema.ts)

[illegible]

```
export const users = pgTable(
  'users',
  {
    id: uuid('id').primaryKey().defaultRandom(),

    email: varchar('email', { length: 255 }).notNull(),
    passwordHash: text('password_hash').notNull(),

    fullName: varchar('full_name', { length: 200 }).notNull(),

    roleId: uuid('role_id')
      .notNull()
      .references(() => roles.id),

    isActive: boolean('is_active').default(true).notNull(),
    lastLoginAt: timestamp('last_login_at'),

    // REMOVED COGNITO COMPATIBILITY:
    // cognitoSubId: uuid('cognito_sub_id').unique(),

    createdAt: timestamp('created_at').defaultNow().notNull(),
    createdBy: uuid('created_by'),

    updatedAt: timestamp('updated_at').defaultNow().notNull(),
    updatedBy: uuid('updated_by'),

    version: integer('version').default(1).notNull(),
  },
  (table) => ({
    emailIdx: uniqueIndex('users_email_idx').on(table.email),
    // REMOVED COGNITO COMPATIBILITY:
    // cognitoSubIdx: uniqueIndex('users_cognito_sub_idx').on(table.cognitoSubId),
  })
);
```

[illegible]

```
export const candidates = pgTable(
  'candidates',
  {
    id: uuid('id').primaryKey().defaultRandom(),
    user_id: uuid('user_id')
```

D. Refactored public.controller.ts (getCognitoSubId)

```
async getCognitoSubId(event: APIGatewayProxyEventV2): Promise<LambdaResponse> {
  try {
    const { body } = parseEvent(event);
    const email = body?.email;
    const emailStr = email && typeof email === 'string' ? email.trim() : undefined;

    if (!emailStr) {
      return response.error(400, { message: 'Email is required and must be a string' });
    }

    /* COGNITO LOOKUP (REMOVED):
    const { getCognitoUserByEmail } = await import('../utils/cognito');
    const userData = await getCognitoUserByEmail(emailStr);
    if (!userData) return response.error(404, { message: 'User not found' });
    */

    // Check if user exists in the local database
    const db = getDb();
    const { users } = await import('../database/schema');
    const dbUsers = await db.select().from(users).where(eq(users.email,
emailStr.toLowerCase())).limit(1);
    const userDbExist = dbUsers.length > 0;

    if (!userDbExist) {
      return response.error(404, { message: 'User not found' });
    }

    return response.success(200, {
      message: 'User retrieved successfully from local database',
      cognitoSubId: dbUsers[0].id,
      userDbExist,
      userData: {
        username: dbUsers[0].email,
        email: dbUsers[0].email,
        name: dbUsers[0].fullName,
      },
    });
  } catch (error) {
    console.error('Error fetching user info:', error);
    return response.error(500, { message: 'Failed to retrieve user info' });
  }
}
```

E. Refactored Auth Service - Login (src/services/auth.service.ts)

[illegible]

F. Refactored Auth Service - Register (src/services/auth.service.ts)

[illegible]


```
});

return {
  userId: user.id,
  candidateId: candidate.id,
  registrationNumber,
  email: user.email,
};
}

async candidateRegister(input: CandidateRegisterInput): Promise<RegisterResult> {
  const existingUser = await userRepository.findByEmail(input.email);
  if (existingUser) {
    throw new ConflictError('User already exists');
  }

  const { roles } = await import('../database/schema');
  const { getDb } = await import('../database/drizzle');
```

G. Refactored Auth Service - Candidate flows (src/services/auth.service.ts)

[illegible]

```

    throw new UnauthorizedError('Your session has expired. Please log in again to continue.');
```

H. Legacy Cognito - Login & SignUp Commands (REMOVED)

[illegible]

```
export const cognitoLogin = async (email: string): Promise<CognitoTokens> => {
  if (config.MOCK_COGNITO) {
    const { userRepository } = await import('../repositories/user.repository');
    const dbUser = await userRepository.findByEmail(email).catch(() => null);

    if (dbUser) {
      const userWithRole = await userRepository.findUserWithRole(dbUser.id).catch(() => null);
      const roleName = userWithRole?.roleName ?? 'candidate';
      const roleGroup = roleName.charAt(0).toUpperCase() + roleName.slice(1) + 's';
      return {
        accessToken: jwt.sign(
          { sub: dbUser.id, email: dbUser.email, 'cognito:groups': [roleGroup] },
          config.JWT_SECRET,
          { expiresIn: 3600 }
        ),
        refreshToken: jwt.sign({ sub: dbUser.id }, config.JWT_SECRET, {
          expiresIn: '30d',
        }),
        idToken: jwt.sign({ sub: dbUser.id, email: dbUser.email }, config.JWT_SECRET, {
          expiresIn: 3600,
        }),
        expiresIn: 3600,
      };
    }
  }

  const user = mockUsers.get(email.toLowerCase());
  if (!user || user.password !== password) {
    throw new Error(
      'The username or password you entered is incorrect. Please check your details and try again.'
    );
  }
  return generateMockTokens(user.email, user.userId);
}
```

```
const params: InitiateAuthCommandInput = {
  AuthFlow: 'USER_PASSWORD_AUTH',
  ClientId: config.COGNITO_CLIENT_ID,
  AuthParameters: {
    USERNAME: email,
    PASSWORD: password,
    SECRET_HASH: computeSecretHash(email),
  },
};
```

```
const command = new InitiateAuthCommand(params);
const result = await cognitoClient.send(command);

return {
  accessToken: result.AuthenticationResult?.AccessToken ?? '',
  refreshToken: result.AuthenticationResult?.RefreshToken ?? '',
  idToken: result.AuthenticationResult?.IdToken ?? '',
  expiresIn: result.AuthenticationResult?.ExpiresIn ?? 3600,
};
};
```

[illegible]

```
export const cognitoRegister = async (
  email: string,
  password: string,
  attributes: Record<string, string> = {}
): Promise<{ userSub: string }> => {
```

```

if (config.MOCK_COGNITO) {
  const { v4: uuidv4 } = await import('uuid');
  const mockId = uuidv4();
  mockUsers.set(email.toLowerCase(), {
    userId: mockId,
    email,
    password,
    roles: ['candidate'],
  });
  return { userSub: mockId };
}

const params: SignUpCommandInput = {
  ClientId: config.COGNITO_CLIENT_ID,
  Username: email,
  Password: password,
  SecretHash: computeSecretHash(email),
  UserAttributes: Object.entries(attributes).map(([Name, Value]) => ({
    Name,
    Value,
  })),
};

const result = await cognitoClient.send(new SignUpCommand(params));
return { userSub: result.UserSub ?? '' };
};
// update using cognito sub id update password and mobile number
export const cognitoUpdateUserAttributes = async (
  cognitoSubId: string,
  attributes: Record<string, string>
): Promise<void> => {
  // MOCK MODE
  if (config.MOCK_COGNITO) {
    for (const user of mockUsers.values()) {
      if (user.userId === cognitoSubId) {
        Object.assign(user, attributes);
      }
    }
    return;
  }

  // 1. Find Cognito Username using sub
  const listRes = await cognitoClient.send(
    new ListUsersCommand({
      UserPoolId: config.COGNITO_USER_POOL_ID,
      Filter: `sub = "${cognitoSubId}"`,
      Limit: 1,
    })
  );
  console.log('step 1: ListUsers result:', listRes);
  const user = listRes.Users?.[0];

  if (!user?.Username) {
    throw new Error(
      'We could not verify your account details. Please try logging in again or contact support.'
    );
  }

  const username = user.Username;

  // 2. Update attributes
  await cognitoClient.send(
    new AdminUpdateUserAttributesCommand({
      UserPoolId: config.COGNITO_USER_POOL_ID,
      Username: username,
      UserAttributes: Object.entries(attributes).map(([Name, Value]) => ({
        Name,
        Value,
      })),
    })
  );
}

```

```
    })),  
  );  
};
```

I. Legacy Cognito Trigger - Custom SMS Sender handler (REMOVED)

```
import {
  buildClient,
  CommitmentPolicy,
  KmsKeyringNode,
} from "@aws-crypto/client-node";

const { decrypt } = buildClient(
  CommitmentPolicy.FORBID_ENCRYPT_ALLOW_DECRYPT
);

const keyArn =
  "arn:aws:kms:ap-south-1:485038482643:key/8801a3cb-af1e-48a9-be43-1b576431cfe7";

interface SendSmsParams {
  mobile: string;
  message: string;
  templateId?: string;
}

interface CognitoEvent {
  triggerSource: string;
  request: {
    code?: string;
    userAttributes: {
      phone_number: string;
    };
  };
  [key: string]: any;
}

const sendSms = async ({
  mobile,
  message,
  templateId,
}: SendSmsParams): Promise<any> => {
  console.log("[SEND_SMS_DISABLED]", {
    mobile,
    message,
    templateId,
  });

  return {
    success: true,
    disabled: true,
  };
};

const handleCustomSMSSender = async (
  event: CognitoEvent
): Promise<CognitoEvent> => {
  const SMS_TRIGGERS = [
    "CustomSMSSender_VerifyUserAttribute",
    "CustomSMSSender_UpdateUserAttribute",
  ];

  if (!SMS_TRIGGERS.includes(event.triggerSource)) {
    console.log(
      "[CustomSMSSender] Skipping -",
      event.triggerSource
    );
  }
}
```

J. Legacy Users DB Schema with Cognito field (REMOVED)

[illegible]

```
export const users = pgTable(
  'users',
  {
    id: uuid('id').primaryKey().defaultRandom(),

    email: varchar('email', { length: 255 }).notNull(),
    passwordHash: text('password_hash').notNull(),

    fullName: varchar('full_name', { length: 200 }).notNull(),

    roleId: uuid('role_id')
      .notNull()
      .references(() => roles.id),

    // NEW: Cognito Sub ID
    cognitoSubId: uuid('cognito_sub_id').unique(),
    // or if you want strict UUID validation:
    // cognitoSubId: uuid('cognito_sub_id').unique().notNull(),

    isActive: boolean('is_active').default(true).notNull(),
    lastLoginAt: timestamp('last_login_at'),

    createdAt: timestamp('created_at').defaultNow().notNull(),
    createdBy: uuid('created_by'),

    updatedAt: timestamp('updated_at').defaultNow().notNull(),
    updatedBy: uuid('updated_by'),

    version: integer('version').default(1).notNull(),
  },
  (table) => ({
    emailIdx: uniqueIndex('users_email_idx').on(table.email),

    // NEW INDEX (important for Cognito lookup)
    cognitoSubIdx: uniqueIndex('users_cognito_sub_idx').on(table.cognitoSubId),
  })
);
```